

YAML-DRIVEN TEMPORAL ORDER MANAGEMENT

Orchestrating Resilient, Decoupled Distributed Transactions

Product & Architecture Overview

The Distributed Saga Challenge

Why Traditional Microservices Architecture Breaks down

THE PROBLEM

- **Fragile Coordination:** Hard to coordinate inventory, payment, and shipping reliably.
- **Orphaned States:** A failure mid-flow leaves inventory locked or payments unrefunded.
- **Complexity Overload:** Developer teams write manual retry loops and custom compensation mechanisms.

THE IMPACT

- **Poor Customer Experience:** Orders get stuck or failed without refund details.
- **Operational Nightmares:** Customer support spends hours matching logs across microservices.
- **Slow Feature Shipping:** Modifying business flows requires massive code refactoring and risk.

The Solution: Temporal Orchestration

Decoupling State Management from Microservice Logic

Reliable Workflows

Temporal guarantees the execution of workflows to the end, even in the event of hardware failures or service outages. State is durable and persistent.

Dynamic Sagas

Provides built-in mechanism for compensation. If step X fails, the engine triggers previous steps' rollback logic (refunds, releases) in exact reverse order.

High Observability

Complete trace of every step, retry count, and delay can be visually audited in the Temporal Web UI. No more parsing endless log messages.

Core Use Case: Order Fulfillment Saga

Executing the complete transaction with automatic rollback safety

Workflow Step	Target Activity	Successful Action	Rollback / Compensation
Reserve Inventory	InventoryActivity	Reserve requested quantities of items	Release reservation
Process Payment	PaymentActivity	Charge user's credit card/wallet	Refund original transaction ID
Create Shipment	ShippingActivity	Generate shipping label & tracking number	(None — no side effect to undo)
Send Notifications	NotificationActivity	Send confirmation & shipping info alerts	(None needed - terminal step)

Event Sourcing: Immutable State as Facts

Every state change is a permanent, ordered event — never an overwrite

What is Event Sourcing?

- State is NEVER stored directly in a row
- Every change raises an immutable DomainEvent
- Current state = replay of all past events
- 13 sealed event types (Java 21):
 - OrderCreatedEvent,
 - ItemAddedEvent,
 - OrderConfirmedEvent,
 - PaymentCompletedEvent,
 - ShipmentCreatedEvent,
 - OrderCancelledEvent...
- Events stored as JSONB in append-only
 - event_store table (no UPDATE, no DELETE)

Aggregate Root: Order.java

- Private fields — NO setters
- command → validate → raise event
- apply(DomainEvent) = ONLY mutation point
- pendingEvents[] drained by repository
- Optimistic lock:
 - UNIQUE(aggregate_id, version)
 - DB constraint + app-level version check
- Snapshot every 50 events:
 - restore snapshot → replay since snapshot
 - worst-case: 50 events replayed, ever

Why vs. Traditional CRUD?

- CRUD overwrites — history is gone
- Event Sourcing: full audit trail always
- PaymentFailedEvent stores failure reason
- InventoryReleasedEvent proves compensation ran
- Debug any point-in-time by replaying to Tv
- Build new read models from old events
- Used alongside Temporal: Temporal manages
 - PROCESS state; Event Store manages
 - DOMAIN state — complementary, not overlapping

Hexagonal Architecture + CQRS

Domain has zero framework dependencies — all external concerns plug in through interfaces

API Layer →

- OrderController (REST)
- WorkflowController (signals/queries)
- GlobalExceptionHandler (RFC 7807 ProblemDetail)
- Bean Validation (@Valid)
- MapStruct DTOs (compile-time)
- Swagger / OpenAPI 3.0
- CorrelationIdFilter → MDC

Application Layer →

- OrderCommandService @Transactional (all writes)
- OrderQueryService
- @Transactional(readOnly=true)
- CQRS: commands return void or new ID only — never state
- Queries never accept command objects

Domain Layer ← (core)

- Order aggregate (zero Spring)
- 13 sealed DomainEvents
- OrderId, CustomerId, Money, OrderItem value objects
- Outbound ports (interfaces): EventStore, SnapshotStore, OrderRepository, WorkflowPort
- Domain exceptions

Infrastructure Layer

- EventStoreAdapter (JDBC)
- SnapshotStoreAdapter
- OrderRepositoryAdapter
- WorkflowPortAdapter (Temporal)
- InventoryActivityImpl (HTTP)
- PaymentActivityImpl (HTTP)
- ShippingActivityImpl (HTTP)
- OrderEventKafkaPublisher

Full System Architecture

5 microservices · Kong API Gateway · Kafka event streaming · Temporal orchestration · 6 isolated databases

Kong Gateway :8000

- Proxy entry point for all client traffic
- Rate limiting (100 req/min)
- Proxy cache (GET 60s)
- correlation-id plugin
- prometheus /metrics
- Routes:
 - /orders/*
 - /workflows/*
 - /inventory/*
 - /payments/*
 - /shipping/*

Microservices

- 📦 order-service :8080
 - REST + Temporal Worker
 - Event Sourcing + CQRS
 - Kafka producer (order.events)
- 🏭 inventory-service :8081
- 💳 payment-service :8082
 - InsufficientFunds (non-retry)
 - CardDeclined (non-retry)
- 🚚 shipping-service :8083
- 🔔 notification-svc :8084
 - Pure Kafka consumer

Kafka :9092

- Topics:
- order.events
 - inventory.events
 - payment.events
 - shipping.events
- notification.requests
- Key = orderId (ordered per partition)
- Outbox Pattern:
DB commit → AFTER_COMMIT → Kafka publish

Temporal :7233

- gRPC endpoint
- Workflow history DB :5433
- Temporal UI :8088

Worker polls task queue:
ORDER_FULFILLMENT_QUEUE

Signals:

cancelOrder(reason)
retryPayment()

Queries:

getCurrentStatus()
getProgress()

Databases (isolated)

- orderdb :5432
 - event_store (JSONB)
 - order_snapshots
- inventorydb :5434
 - reservations
- paymentdb :5435
 - transactions
- shippingdb :5436
 - shipments
- temporal-pg :5433
- kong-pg

The Leap: Hardcoded vs. Dynamic Configuration

Moving business logic out of the codebase and into YAML config

BEFORE: HARDCODED LOGIC

- Inflexible Flows: Execution sequence was hard-coded in Java (reserve -> pay -> ship).
- Recompile Required: Change in a step timeout, retry strategy, or workflow order required a full code change, review, and redeployment.
- Manual Sagas: Compensation checks and custom exceptions were hardcoded per step.

AFTER: DYNAMIC YAML ENGINE

- Declarative Configuration: Steps, order of execution, timeouts, and rollbacks are defined in a simple YAML config file.
- Zero Re-deploys: Modify step execution sequence, timeouts, or retries at runtime without modifying a single line of Java code.
- Automatic Rollbacks: The engine dynamically builds a history stack and calls correct compensations from the configuration.

Decoupled Logic: YAML Configuration

Standardized and clean YAML blueprint for our business rules

workflow-config.yml

```
name: "StandardOrderFulfillment"
steps:
  - name: "RESERVE_INVENTORY"
    timeoutSeconds: 30
    maxAttempts: 3
    compensationActivity: "InventoryActivity"
    compensationMethod: "releaseInventory"

  - name: "PROCESS_PAYMENT"
    timeoutSeconds: 60
    maxAttempts: 2
    compensationActivity: "PaymentActivity"
    compensationMethod: "refundPayment"

  - name: "CREATE_SHIPMENT"
    timeoutSeconds: 30
    maxAttempts: 3
```

How the Engine Works Under the Hood

Combining Java 21 Records, Jackson, and Temporal stubs

1. Parsing

YamlWorkflowLoader reads the YAML config, mapping it directly into immutable Java 21 Record classes for zero-overhead validation and parsing.

2. Loop Execution

The workflow iterates through the parsed configuration list. Temporal ActivityOptions (timeouts, retries) are dynamically injected per step.

3. Stack Rollbacks

Successful steps are pushed onto a LIFO stack. Upon any failure, the stack is popped, and compensation methods are dynamically executed.

Testing & Resiliency Verification

Ensuring dynamic logic operates reliably in all runtime failure conditions

IN-MEMORY TEST ENVIRONMENT

- Isolated Tests: `OrderFulfillmentWorkflowTest` utilizes `TestWorkflowEnvironment` for virtual time testing.
- Simulated Outages: Test doubles mock failures (e.g. `CardDeclinedException`, slow latency) to verify compensating rollbacks.
- Fast Executions: Fast-forwards time automatically (5-minute timeouts resolved in milliseconds).

LIVE INTEGRATION TESTS (IT)

- Live Connection: `OrderFulfillmentLiveWorkflowIT` connects to a live running local Temporal instance.
- Visual Auditing: Developer can physically inspect the step-by-step saga trace live inside the Temporal Web UI.
- Production Readiness: Validates correct YAML deserialization against real gRPC server calls.

Business Outcomes & Next Steps

Strategic advantages of declarative transaction orchestration

VALUE DELIVERED

- **Faster Time-to-Market:** Business flows adjusted instantly via configuration changes.
- **Decreased Operational Overhead:** Auto-recovery of partial transactions lowers customer support tickets.
- **Developer Velocity:** Standardized, dynamic Saga logic removes boilerplate code from codebase.

ROADMAP & EVOLUTION

- **A/B Testing:** Support loading different YAML profiles based on customer segments.
- **UI Configurator:** Build a low-code canvas interface that generates the workflow-config.yml directly.
- **Observability dashboard:** Extend Grafana panels to count step-level latency based on YAML config.